Homework 2: Test Generation and Improvement Using Codex

CS 59300

November 3, 2026

1 Test Generation Approach

Folio AI Research Assistant is a Flask application for searching papers, maintaining a SQLite library, extracting PDF text, and generating summaries and answers. Codex examined the existing source and README and generated automated tests with Python's `unittest`, Flask's test client, temporary databases, and mocked external HTTP. The user requested that Codex complete HW2 against the HW1 project, use ARIS for the report and README, and follow the assignment requirements. The original repository already contained 11 HW1 workflow test methods; these were retained. The initial HW2 addition comprised four methods: health, invalid search, paper save and library filtering, and missing paper. The initial measurement includes both the existing and newly generated tests. No live credentials, network calls, or personal library data were used.

2 Initial Test Evaluation

All 15 initial tests passed. Coverage.py 7.13.1 measured 73% combined statement and branch coverage for the Python backend; 149 of 680 statements were missed. The baseline exercised PDF upload and model workflows through the existing suite, but search caching, input validation, authentication, and Crossref parsing were weakly covered. The new health check asserted the complete response, but it was a smoke check and did not establish broader functional correctness. The new unknown-paper test overlapped somewhat with the pre-existing deletion test. Review against the README and API validation found no demonstrated invalid initial test, though checking implementation alone cannot establish product intent. No application bugs were detected in this initial run; passing tests cannot rule out defects.

3 Iterative Test Improvement

The user supplied the assignment and the HW1 repository, rather than separate requirements for each round. The following are *summaries of the agent's actual self-directed guidance*, not verbatim user prompts. The initial direction was to inspect the application and produce executable tests for main and error paths. Codex then inspected the initial coverage report and chose two distinct follow-up strategies:

1. **Round 1 — coverage and state feedback.** Focus on uncovered metadata validation and search caching. Three methods check invalid fields and search parameters, verify invalid saves leave the library empty, and verify cached search results reflect a subsequently saved paper without another upstream call. All 18 tests passed; combined coverage rose to 78%.
2. **Round 2 — requirements and boundaries.** Examine access control, Crossref mapping, and analysis without full text. Four methods test Basic authentication and the health exception, cross-site rejection, input validation and missing PDF responses, and HTML/author/year mapping from a mocked Crossref response. All 22 tests passed; coverage rose to 82%.

No application code was changed to make tests pass. Test expectations were checked against the README and the application behavior; Codex authored and reviewed the tests. There were no significant manual edits by the user in these rounds. Full run and coverage outputs are in `evidence/`; ARIS's deterministic evidence checker verified the reported run counts and percentages against those files.

4 Final Evaluation

Stage	Test methods passed	Missed statements	Combined coverage
Initial	15/15	149/680	73%
Round 1	18/18	125/680	78%
Final	22/22	101/680	82%

The denominator and configuration were held constant: Python backend only, with branch measurement enabled. Coverage.py's combined percentage includes branch opportunities and is not a pure line-coverage percentage. Final module coverage was 88% for the Flask API, 94% for the database, 79% for LLM integration, 70% for PDF processing, and 77% for search. The increase supports broader execution, not proof of correctness or fault detection. No application bugs were detected by the final suite. Tests compare observable responses and state to expected behavior, including a cache-state change and refusal to analyze a Crossref record without an attached PDF. No mutation testing was performed, so sensitivity to seeded faults was not measured.

Round 1 was effective at exercising validation and cache behavior; round 2 improved coverage of security and external data mapping. A simple health check remained useful as a smoke test but was less informative than stateful assertions. PDF download failures, some model error paths, concurrent jobs, browser JavaScript, and actual upstream services remain incompletely tested. Mocks make results deterministic but cannot validate real provider behavior, PDF extraction across all documents, model answer quality, or the visual interface.

5 Reflection

Codex understood the principal Flask routes and generated isolated, repeatable checks quickly, especially for API responses and library persistence. Its initial selection underrepresented validation and service integration boundaries; a passing smoke test alone did not demonstrate useful functional coverage. Coverage feedback pointed to specific omissions, while the README and route behavior helped select meaningful assertions. These results show that agent-generated tests benefit from execution feedback and human review of expected behavior. Coverage is a useful map of execution, but it is not a substitute for a specification or evidence that faults would be caught.

6 Repository and Reproduction

Repository: https://github.com/lhxxh/CS59300_hw1. Original tests are in `tests/test_workflow.py`; new tests are in `tests/test_hw2_api.py`. Run the following in a Python 3.12 or 3.13 environment from the repository root:

```
python -m pip install -r requirements.lock
python -m pip install 'coverage>=7,<8'
python -m unittest discover -s tests -v
python -m coverage run --branch --source=research_assistant \
  -m unittest discover -s tests -v
python -m coverage report -m
```

The recorded results are snapshots from Python 3.13.12 on the same source revision; timings and coverage details may vary by environment. Tests use temporary stores and mock network services. The README includes the same commands.